

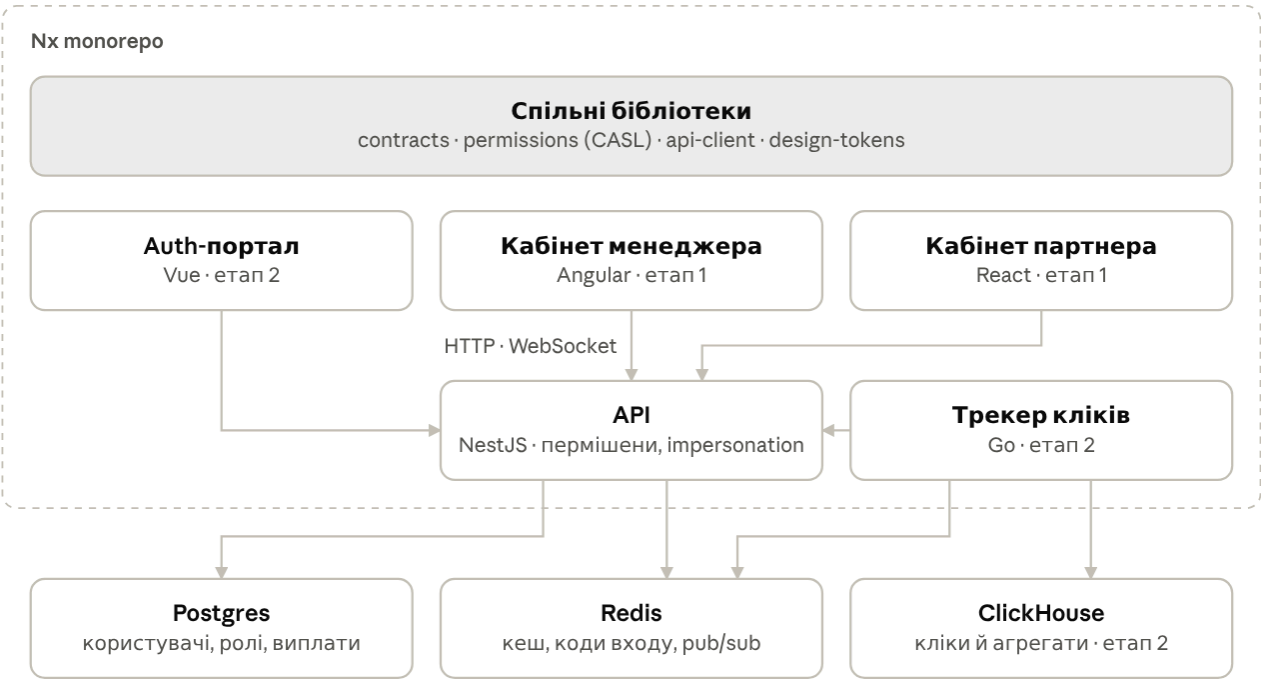
Affiliate Platform Portfolio — план реалізації

Oct 4, 2026 · @Mikhailo

Огляд

Портфоліо — це affiliate-платформа в одному Nx-монорепо, де ядро — гнучкі пермішени менеджерів і вхід менеджера в кабінет партнера. Проект будується в чотири етапи, і після кожного репо готове до показу.

Один Nx-монорепо: три фронти, API на Nest і Go-трекер



архітектура · 3 фронти, 2 бекенди, 3 сховища

Фронти ходять в API на Nest; Go-трекер приймає трафік, пише кліки в ClickHouse і передає події в API через Redis, а API читає з ClickHouse агрегати для дашбордів.

Етап	Що додає	Реліз
0	Nx-воркспейс, tooling, дизайн-токени, CI	—
1	MVP: auth, пермішени, impersonation, обидва кабінети, деплой	v0.1

Етап	Що додає	Реліз
2	Go-трекер, ClickHouse, Vue auth-портал, навантаження	v0.2
3	WebSockets: нотифікації, live-статистика	v0.3

Етап 0. Nx-воркспейс і фундамент

Результат етапу — порожній, але робочий воркспейс: усі застосунки збираються, лінтяться й тестуються однією командою, а CI зелений з першого коміту. Go і Vue додаються лише на Етапі 2.

Кроки

- Створити воркспейс: `pnpm dlx create-nx-workspace@latest affiliate-platform --preset=apps --pm=pnpm`. Увімкнути `pnpm` через `corepack enable` і зафіксувати його версію в полі `packageManager` у `package.json`, а Node LTS — у `.nvmrc`.
- Усі команди Nx далі запускаються через `pnpm nx` (без `npm` і `npx`). Додати плагіни: `pnpm nx add @nx/nest, @nx/angular, @nx/react, @nx/vite, @nx/js, @nx/playwright`.
- Згенерувати застосунки (до кожного генератора додати `--unitTestRunner=vitest`):
 - `nx g @nx/nest:app apps/api`
 - `nx g @nx/angular:app apps/manager-web --bundler=esbuild`
 - `nx g @nx/react:app apps/partner-web --bundler=vite`
- Згенерувати бібліотеки (`nx g @nx/js:lib`, для UI — генератор відповідного фреймворку) і одразу проставити теги з таблиці нижче.
- Межі модулів без ESLint: свій скрипт `tools/check-boundaries.ts` на `@nx/devkit` (`createProjectGraphAsync`) перевіряє кожне ребро графа за таблицею правил нижче і запускається як таргет `pnpm nx run workspace:check-boundaries`. Для миттєвого фідбеку в редакторі — правило `noRestrictedImports` у Biome через `overrides` по папках кожного scope. Angular-шаблони перевіряє сам компілятор: `strictTemplates` і `extendedDiagnostics` з рівнем `error`.
- Тести: Vitest замість Jest у всіх TS-проєктах і бібліотеках; для NestJS — плагін `unplugin-swc`, бо `esbuild` не генерує метадані декораторів, потрібні для DI; спільний базовий конфіг Vitest у корені воркспейсу. Tooling: Biome для

лінтингу й форматування всього TS/JS-коду (ESLint і Prettier у проєкті немає), один biome.json у корені, Husky + lint-staged, commitlint (Conventional Commits), .editorconfig.

- 7. Docker лише для інфраструктури в розробці: Postgres і Redis через Docker Compose (ClickHouse додається на Етапі 2). Застосунки запускаються локально через npx nx serve і в контейнери не пакуються.
- 8. Дизайн: libs/shared/design-tokens з CSS-змінними і Tailwind preset; підключити його в обидва фронти; ініціалізувати spartan/ui в Angular і shadcn/ui в React.
- 9. CI-каркас у GitHub Actions: nrwl/nx-set-shas , кеш npx, nx affected -t lint test build check-boundaries.

Бібліотеки і теги

Бібліотека	Теги	Що всередині
shared/contracts	scope:shared, type:util	DTO, enums, zod-схеми
shared/permissions	scope:shared, type:util	список пермішенів, фабрика CASL ability
shared/api-client	scope:shared, type:data-access	клієнт, згенерований з OpenAPI
shared/design-tokens	scope:shared, type:ui	CSS-змінні, Tailwind preset
api/auth , api/users , api/offers , api/partners , api/payouts , api/audit	scope:api, type:feature	модулі NestJS
manager/feature-* , manager/data-access , manager/ui	scope:manager	кабінет менеджера
partner/feature-* , partner/data-access , partner/ui	scope:partner	кабінет партнера

Правила залежностей

Тег	Може імпортувати
scope:shared	scope:shared

Тег	Може імпортувати
scope:api	scope:api, scope:shared
scope:manager	scope:manager, scope:shared
scope:partner	scope:partner, scope:shared
type:feature	feature, ui, data-access, util
type:data-access	data-access, util
type:ui	ui, util
type:util	util

Готово, коли

- ☐ `nx run-many -t lint test build` проходить локально
- ☐ Імпорт з `manager/*` у `partner/*` падає на `check-boundaries` і підсвічується Biome в редакторі
- ☐ `docker compose up` піднімає Postgres і Redis
- ☐ Обидва фронти показують кнопку в однаковому стилі з одних токенів
- ☐ CI зелений на першому PR

Етап 1. MVP: два кабінети, пермішени, impersonation

Результат етапу — задеплоєне демо, де менеджери з різними ролями бачать різний інтерфейс, можуть увійти в кабінет партнера, а партнер бачить свою статистику. Пермішени — центральна фіча, тому їх робимо одразу після auth, до будь-яких бізнес-модулів.

1.1 База даних (Postgres + Prisma)

1. Моделі: `users` (тип `manager` | `partner`, статус), `roles`, `permissions`, `role_permissions`, `partners` (з `assigned_manager_id`), `offers`, `tracking_links`, `clicks`, `conversions`, `payouts`, `refresh_tokens`, `audit_log`.
2. Міграції через Prisma Migrate, seed-скрипт як окремий Nx-таргет `nx run api:seed`.
3. На цьому етапі кліки пишуться в Postgres. На Етапі 2 вони переїдуть у ClickHouse — ця еволюція сама по собі гарна історія для README.

1.2 Auth

1. Логін за email + пароль, хешування Argon2.
2. Access JWT на 15 хвилин, refresh-токен у httpOnly cookie з ротацією та reuse detection.
3. На цьому етапі форми логіну живуть у кожному кабінеті; на Етапі 2 їх замінить Vue auth-портал.
4. Rate limit на логін (@nestjs/throttler).

1.3 Пермішени (ядро проєкту)

1. У shared/permissions описати пермішени у форматі resource:action :
offers:read , offers:write , partners:read , partners:approve ,
partners:impersonate , payouts:approve , stats:read , roles:manage ,
audit:read .
2. Там же — функція defineAbilityFor(user) на CASL, спільна для бекенду і обох фронтів.
3. Data-level умови: account manager працює лише зі своїми партнерами (assigned_manager_id = user.id).
4. Nest: декоратор @RequirePermissions() + guard; фільтрація даних через accessibleBy() з @casl/prisma .
5. Endpoint /me/permissions замість повного списку прав у JWT.
6. UI управління ролями в кабінеті менеджера: створення ролі, матриця чекбоксів пермішенів, призначення ролі менеджеру.
7. Seed-ролі: Admin (усе), Account manager (свої партнери, impersonation), Finance (виплати і статистика), Viewer (лише читання).

1.4 Вхід менеджера в кабінет партнера (impersonation)

1. Окремий пермішен partners:impersonate ; account manager може входити лише в кабінети своїх партнерів.
2. Менеджер тисне «Увійти як партнер» → POST /auth/impersonate/:partnerId повертає одноразовий код на 60 секунд (у Redis).
3. partner-web відкривається в новій вкладці з цим кодом і міняє його на токен. Сам токен ніколи не передається в URL.
4. Токен партнера з claim act: { sub: managerId } (за зразком RFC 8693), живе 30 хвилин, без refresh-токена.
5. В partner-web — постійний банер «Ви в кабінеті X від імені менеджера Y» і кнопка «Завершити».

6. Під час impersonation блокуються чутливі дії: зміна пароля, email і платіжних реквізитів (guard перевіряє наявність `act`).
7. Audit log: початок і кінець сесії, а кожна write-дія записується з обома акторами (партнер + менеджер).

1.5 Бізнес-модулі API

1. Offers: CRUD, статус active/paused, payout за конверсію.
2. Partners: реєстрація, approve/block, призначення менеджера.
3. Tracking: генерація лінків, endpoint редіректу з записом кліку, postback для конверсій.
4. Stats: агрегати по днях, оферах і партнерах.
5. Payouts: запит партнера → approve фінансистом → paid.
6. Swagger на `/docs` , з нього генерується `shared/api-client` (окремий Nx-таргет).

1.6 Кабінет менеджера (Angular)

1. Standalone-компоненти, signals, lazy-роути на кожну feature-бібліотеку.
2. Пермішени: route guards, pipe `able` , меню фільтрується під права.
3. Екрани: офери, партнери (з кнопкою «Увійти як партнер»), виплати, статистика з фільтрами, ролі, audit log.

1.7 Кабінет партнера (React)

1. React Router, TanStack Query поверх `shared/api-client` .
2. Екрани: дашборд з графіком, доступні офери, генерація трекінг-лінків, виплати, профіль.
3. Обробка impersonation: обмін коду, банер, заблоковані чутливі дії.

1.8 Тести, CI/CD і демо

1. Unit-тести (Vitest) на `shared/permissions` і guards; матричний тест «роль × endpoint → статус».
2. E2E на Playwright: менеджер без `payouts:approve` не бачить кнопку і отримує 403; impersonation від початку до кінця.
3. Testcontainers для інтеграційних тестів API з реальним Postgres.
4. CI: lint, test, build, e2e для affected; спосіб деплою демо визначимо окремо.
5. Демо-дані та доступи: 4 менеджери з різними ролями, 2-3 партнери, статистика за 90 днів.

Готово, коли

- ☐ Демо доступне за посиланням і тестові логіни вказані в README
- ☐ Кожна роль менеджера бачить лише свої розділи і дані
- ☐ Impersonation працює, чутливі дії заблоковані, дії видно в audit log
- ☐ Прямий запит до API без права повертає 403
- ☐ CI проходить на кожному PR

Етап 2. Go-трекер, ClickHouse і Vue auth-портал

Результат етапу — високонавантажений шлях кліків винесений у Go, статистика рахується в ClickHouse, а вхід для всіх ролей — через єдиний auth-портал. В README з'являються цифри навантажувальних тестів.

2.1 ClickHouse

1. Додати ClickHouse у Docker Compose; міграції SQL-файлами (наприклад, golang-migrate).
2. Таблиці `clicks` і `conversion_events` на MergeTree: партиціювання по місяцю, сортування `partner_id`, `offer_id`, `ts`, TTL для сирих подій.
3. Materialized view з `SummingMergeTree` для погодинних агрегатів (кліки, унікальні, конверсії, дохід).
4. Статуси конверсій і виплати лишаються в Postgres.
5. Nest-модуль Stats переключається на читання агрегатів з ClickHouse; пермішени і data-level фільтри працюють як і раніше.

2.2 Go-трекер (`apps/tracker`)

1. Підключити `@nx-go/nx-go`, щоб build, test і lint Go-сервісу були звичайними Nx-таргетами і потрапляли в `nx affected`.
2. `GET /c/:linkId`: валідація лінка з кешем у Redis, запис кліку в буфер, 302 на лендінг з `click_id`.
3. `GET /postback`: конверсія за `click_id`, подія в ClickHouse і в Nest (через Redis) для запису статусу в Postgres.
4. Буфер на каналах і горутинах: batch insert кожні N секунд або N подій, graceful shutdown зі скиданням буфера.
5. Prometheus-метрики (RPS, latency, розмір буфера) і health-check.
6. Endpoint редіректу в Nest з Етапу 1 видаляється.

2.3 Auth-портал на Vue (apps/auth-web)

1. `pnpm nx add @nx/vue`, застосунок на Vite з тим самим Tailwind preset; теґ `scope:auth`.
2. Екрани: логін, реєстрація партнера, відновлення пароля.
3. Після входу — редірект у потрібний кабінет за типом користувача через одноразовий код (той самий механізм, що в impersonation).
4. Форми логіну з кабінетів видаляються.

2.4 Навантаження

1. Скрипт k6: потік кліків на трекер і генератор, що заливає мільйони подій у ClickHouse.
2. У README: RPS і p95 latency трекера, час відповіді дашборду на цих даних, порівняння з Postgres-версією з Етапу 1.

Готово, коли

- ☐ Трекер не втрачає подій при перезапуску
- ☐ Дашборди обох кабінетів читають дані з ClickHouse
- ☐ Вхід для всіх ролей тільки через auth-портал
- ☐ Цифри навантаження в README

Етап 3. Realtime на WebSockets

Результат етапу — нотифікації, live-статистика і миттєве оновлення прав без перелогіну. Це демонстрація навички на реальних сценаріях, а не критична функціональність.

Кроки

1. Nest-gateway на Socket.IO; auth на handshake тим самим JWT (включно з impersonation-токенами).
2. Rooms: `partner:{id}`, `managers`, `manager:{id}`; підписка на room перевіряється тим самим CASL ability.
3. Go-трекер публікує події в Redis pub/sub; Nest агрегує їх і шле дельту раз на 1–2 секунди, а не кожен клік.
4. Redis adapter для Socket.IO; дві репліки API локально на різних портах, щоб показати горизонтальне масштабування.
5. Нотифікації партнеру: конверсію підтверджено, виплату відправлено, офер поставлено на паузу.

6. Нотифікації менеджеру: новий партнер чекає на approve, запит на виплату, аномальний сплеск кліків.
7. Зміна ролі менеджера → подія `permissions:changed` → клієнт перечитує `/me/permissions` і перебудовує UI.
8. Двоспрямовий канал: клієнт підтверджує прочитання нотифікацій, `presence` «менеджер онлайн» у кабінеті партнера.
9. Фронт: в Angular — сервіс з RxJS-стрімами, у React — кастомний хук з інвалідацією TanStack Query.

Готово, коли

- ☐ Партнер бачить ріст кліків під час k6-тесту в реальному часі
- ☐ Партнер не може підписатися на чужу room
- ☐ Зміна ролі застосовується без перелогіну
- ☐ Події доходять при двох репліках API

README і презентація репозиторію

Рекрутер або рев'юер витрачає на репо 2-3 хвилини, тому README оновлюється наприкінці кожного етапу, а не лише в кінці.

1. Перший екран: одне речення про проєкт, посилання на live demo, таблиця тестових логінів по ролях, GIF або скріншот обох кабінетів.
2. Діаграма архітектури (Mermaid прямо в README).
3. «Чому такий стек»: по одному реченню на кожен технологію — це знімає враження «зоопарку».
4. Розділ про пермішени і impersonation як ключову фічу: модель, де вони перевіряються, захист чутливих дій.
5. Quick start: `pnpm i`, `docker compose up`, `nx run-many -t serve` — не більше трьох команд.
6. Цифри навантажувальних тестів (після Етапу 2).
7. ADR-и в `docs/adr/`: чому Nx, чому CASL, чому ClickHouse, чому Go для трекера, чому WebSockets, а не SSE.
8. Граф залежностей з `nx graph` як картинка — наочно показує module boundaries.
9. Чиста історія: Conventional Commits, робота через PR з описом навіть самотійно, теги релізів `v0.1` / `v0.2` / `v0.3` на кінець кожного етапу.